

Secrets and Keys Management

Purpose and security boundary

This project uses one backend-held credential to access an OpenAI-compatible provider. For the default Virginia Tech Advanced Research Computing (ARC) profile, that credential is the researcher's personal ARC LLM API key.

The browser must never receive, request, persist, or display this key. Browser provider profiles contain only non-secret settings such as the endpoint, model, reasoning effort, limits, search mode, tool allowlist, and TLS preference. The FastAPI process reads the key from its environment and adds the authorization header only after provider-host validation.

Do not place a real credential in source files, Markdown, prompts, issues, chat transcripts, tests, fixtures, snapshots, logs, telemetry, generated reports, shell commands, screenshots, or browser storage. Never ask another person or a coding agent to paste or reveal a key.

Secret inventory

Name or class	Status	Purpose	Approved source
LLM_API_KEY	Secret; currently implemented	Authorizes provider model, file, and related API requests	Process environment, root .env for Compose, or explicitly selected secrets/local.env
ALLOWED_PROVIDER_HOSTS	Non-secret	Restricts credential-bearing requests to approved hosts	Environment or safe defaults
MAX_UPSTREAM_OPERATIONS	Non-secret	Bounds concurrent provider work; defaults to 4	Environment or safe defaults
PROVIDER_TIMEOUT_SECONDS	Non-secret	Provider request timeout; defaults to 120	Environment or safe defaults
MAX_CONTEXT_CHARACTERS	Non-secret	Local context budget; defaults to 120000	Environment or safe defaults
Browser provider profile	Non-secret	Base URL, model, reasoning effort, output limit, timeout, search/tool configuration, and TLS preference	Browser IndexedDB
Future MCP bearer tokens and stdio credentials	Secret; not implemented yet	Authorizes configured MCP services	Environment variables referenced by configuration, never literal YAML values

LLM_API_KEY is the only credential currently required for ARC. Do not invent additional ARC secret variables or copy the key into multiple variables.

Obtain an ARC API key

ARC currently provides the OpenAI-compatible base URL <https://llm-api.arc.vt.edu/api/v1>. Virginia Tech students, faculty, and staff create a personal key at <https://llm.arc.vt.edu> under User profile > Settings > Account > API keys. ARC states that keys are unique to each user, must remain confidential, and must not be shared.

The application sends the key as an Authorization: Bearer header. The checked-in default profile uses Kimi-K3-thinking-max-legacy-tool-calling and enables the model-selected server:websearch tool. ARC can add, remove, or rename models, so use the application's model-discovery feature and re-check the ARC documentation before changing the default alias.

Authoritative reference: ARC LLM API documentation. Configuration details in this guide were last verified on 2026-08-27.

When to connect ARC

G02 established the backend credential boundary, provider profiles, model discovery, connectivity testing, and streamed provider calls. The project is already past that goal, so ARC may be connected now for explicit live smoke tests; no later goal is required to let Codex start the application or exercise the existing backend through the normal project commands.

Keep routine coding and automated verification credential-free and mocked. G05-S5.1 is the point at which live ARC search behavior becomes integration-specific acceptance work, while G05-S5.3 covers MCP connectivity and uses separate service credentials if a chosen MCP server requires them. Connecting the application to ARC does not replace the model that powers Codex or give Codex an independent ARC account: Codex can use ARC only through an authorized project process that inherits LLM_API_KEY.

Configure Docker Compose

Docker Compose is the production-shaped local runtime and automatically reads the root .env file for interpolation.

1. Copy .env.example to .env without changing .env.example.
2. Edit .env locally and set LLM_API_KEY=<ARC_API_KEY> using the actual personal key.
3. Leave ALLOWED_PROVIDER_HOSTS at its checked-in default unless another approved provider is intentionally required.
4. Start or recreate the services with `docker compose up --build` so the API container receives the current environment.
5. Open the loopback web URL and use the provider connectivity test. Never paste the key into the browser.

The root .env is ignored by Git and excluded from the Docker build context. Compose passes its value into the API container environment; it does not use Docker Secrets. A local user or administrator with Docker inspection privileges may be able to view container environment variables. Keep the machine and Docker daemon access restricted.

An alternative local file may be stored as secrets/local.env and selected explicitly:

```
docker compose --env-file secrets/local.env up --build
```

```
docker compose --env-file secrets/local.env up --build
```

Compose does not read secrets/local.env unless --env-file is supplied. Do not pass the key itself as a command-line argument.

Configure native development

The native development scripts do not load .env or secrets/local.env. They inherit variables from the shell that launches them.

Windows PowerShell

Use a hidden prompt so the key is not echoed or placed in command history. The child API process still receives a plain environment string because HTTP authentication requires it.

```
$SecureArcKey = Read-Host "ARC API key" -AsSecureString
$SecretPointer = [Runtime.InteropServices.Marshal]::SecureStringToBSTR($SecureArcKey)
try {
    $env:LLM_API_KEY = [Runtime.InteropServices.Marshal]::PtrToStringBSTR($SecretPointer)
} finally {
    [Runtime.InteropServices.Marshal]::ZeroFreeBSTR($SecretPointer)
}
./scripts/dev.ps1
Remove-Item Env:LLM_API_KEY -ErrorAction SilentlyContinue
```

The cleanup command runs only after the development command exits. Close the shell after use if there is any doubt about inherited environment state.

POSIX shell

```
IFS= read -r -s LLM_API_KEY
printf '\n'
export LLM_API_KEY
./scripts/dev.sh
unset LLM_API_KEY
```

Do not use an inline assignment containing the real key, because it can enter shell history, terminal transcripts, process inspection, or copied support output.

Protect local secret files

- Use a dedicated personal ARC key; never share another researcher's key or reuse it for unrelated systems.
- Limit `.env` or `secrets/local.env` access to the owning user. On POSIX systems, use `chmod 600 <file>`. On Windows, inspect permissions with `icacls <file>` and remove unintended user/group access using approved local IT practices.
- Keep secret files only in the documented ignored locations. Avoid synced folders, shared drives, email, issue attachments, and unencrypted backups.
- Do not store credentials in a VS Code setting, launch configuration, `.envrc`, MCP YAML, browser password field, IndexedDB, localStorage, or frontend build variable. Values prefixed with `VITE_` are compiled for browser use and are never appropriate for secrets.
- Do not disable TLS verification for ARC. The browser profile's TLS override is a visible diagnostic escape hatch, not a credential workaround.
- Clear copied keys from the clipboard after setup and avoid screen sharing while a key-management page or secret file is open.
- Treat crash dumps, debugger captures, environment listings, PowerShell transcripts, CI logs, Docker inspection output, and support bundles as possible disclosure channels.

Safe coding, testing, and Codex use

The normal unit, component, and browser suites use mocked providers and require no live key. Keep those suites deterministic and offline.

Live ARC checks must remain explicit and opt-in. A test or coding-agent command may inherit `LLM_API_KEY` from the environment, but it must never print the environment, request headers, raw provider bodies, or a fully expanded Compose configuration. Application configuration does not make ARC the model powering Codex and does not automatically grant Codex direct access to ARC.

Before authorizing a live check:

1. Confirm the command reads `LLM_API_KEY` from the environment rather than a flag or source file.
2. Confirm logging is metadata-only and errors are sanitized.
3. Use a low-cost, non-sensitive test prompt.
4. Verify the target resolves to the allowed ARC HTTPS host.
5. Remove the process-scoped variable and stop/recreate containers when finished.

Never run `docker compose config`, `Get-ChildItem Env:`, `env`, `set`, or diagnostic commands that dump full process/container environments into a transcript while a real key is loaded. If Compose structure must be inspected, override `LLM_API_KEY` with an empty value in that verification process.

Rotation and revocation

Rotate a key when its intended lifetime ends, a machine or account changes hands, permissions are uncertain, or any possible disclosure occurs.

1. Revoke the affected key through the ARC account API-key settings immediately.
2. Create a replacement personal key.
3. Replace the value in the approved local secret location or launching environment.
4. Stop and recreate all API processes and containers that inherited the old value.
5. Remove stale copies from environment variables, clipboard managers, local secret files, approved secret managers, and backups where feasible.
6. Run a sanitized connectivity test and monitor for unexpected failures or use.

If a key appears in Git, a chat, a log, a screenshot, a generated report, or any shared system, treat it as compromised even if the content is deleted. Revocation comes first. Then determine the disclosure scope, inspect repository history and artifacts without redisplaying the key, clean history through a reviewed recovery procedure, notify collaborators who may have fetched it, and follow ARC/VT incident-reporting requirements where unauthorized use is suspected.

Repository safeguards and their limits

- `.gitignore` excludes approved secret files before they are tracked. It does not protect a file that was already committed and does not inspect file contents.
- `.dockerignore` prevents local secret material from being sent in the Docker build context. It does not hide environment variables injected into a running container.
- `.gitattributes` marks secret paths `export-ignore` as archive defense in depth. It does not block `git add`, and secret files must not be tracked in the first place.
- `scripts/security.ps1` and `scripts/security.sh` scan repository text for selected credential formats and verify required exclusion rules. Pattern scanning is not exhaustive and cannot prove that arbitrary content is non-secret.
- `.env.example` is intentionally tracked and must contain names, safe defaults, and blank placeholders only.

Before committing, run the security script and review staged changes by filename and content. Never use `force-add` to bypass an ignore rule for a credential-bearing file.